Software Architecture Document (SAD)

Project: Insurance Workflow Automation System
Version: 1.0
Authors: Sanjith Suresh Kumar (Lead), Sakaray Vishal, Samarth A, Sampreeth Jaladi
Date: 09-09-2025
Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	09-09-2025	Sanjith Suresh Kumar	Initial architecture document
0.8	01-09-2025	Samarth A	Component & sequence diagrams added

Approvals

Role	Name	Signature / Date
Project Lead	Sanjith Suresh Kumar	—
DevOps Lead	Sampreeth Jaladi	—
QA Lead	Sakaray Vishal	—

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Insurance Workflow Automation System (IWAS).

1.2 Scope

Covers user services: quote generation, underwriting, policy issuance, claims processing, and workflow orchestration.

1.3 Audience

Developers, QA engineers, security auditors, instructors, and maintenance teams.

1.4 Definitions

IWAS, BPMN, KYC, RBAC, ADR, PCI-DSS, TLS, SLA, CI/CD.

2. Document Overview

2.1 How to Use This Document

Provides architectural deliverables including UML diagrams, ADRs, threat models, and API design.

2.2 Related Documents

SRS, STP, RTM.

3. Architecture

3.1 Goals & Constraints

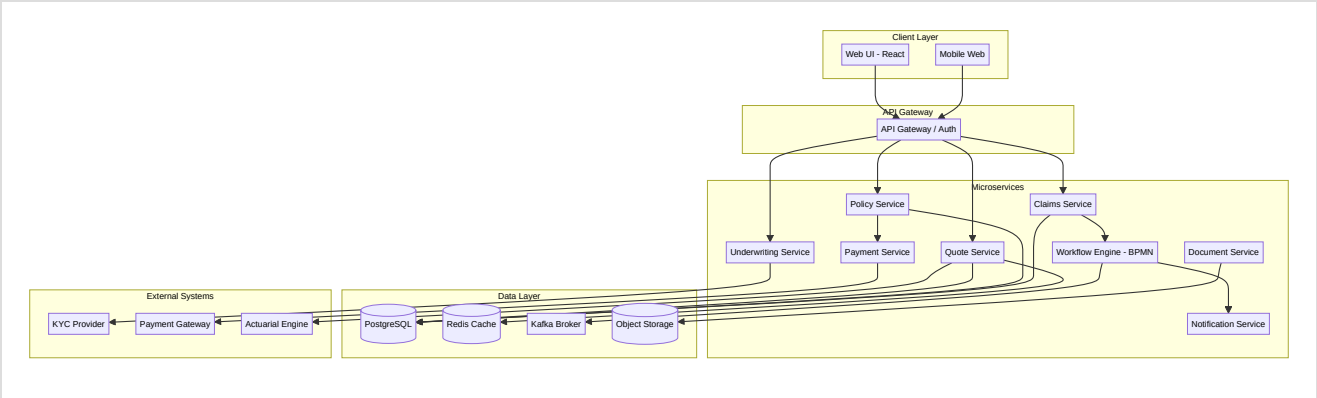
Goals: Secure, reliable, 99.9% availability, scalable to 1M+ policies.

Constraints: IRDAI & GDPR compliance, PCI-DSS for payments, BPMN 2.0 workflow standards, cloud-native deployment.

3.2 Stakeholders & Concerns

- Customers: security, availability, easy claims
- Bank Ops / Insurers: maintainability, SLA tracking
- Regulators: compliance, audit trails
- Developers: modularity, testability, CI/CD

3.3 Component (UML) Diagram



3.4 Component Descriptions

Component	Responsibility	Technology
Web UI	Customer / agent / admin interfaces	React, TypeScript
API Gateway	Routing, auth, rate limiting	Kong / Spring Cloud Gateway
Quote Service	Premium calculation and quote generation	Java Spring Boot
Underwriting Service	Risk scoring, rule engine, KYC integration	Java Spring Boot + Drools
Policy Service	Issuance, renewal, endorsement	Java Spring Boot
Claims Service	Claim registration, tracking, settlement	Java Spring Boot
Workflow Engine	BPMN orchestration, task routing	Camunda / Flowable
Notification Service	Email, SMS, in-app alerts	Node.js + Kafka consumer
Payment Service	Premium collection, refunds	Java + Razorpay SDK
Document Service	Policy PDFs, claim docs	Python + S3
PostgreSQL	Primary transactional data store	PostgreSQL 15
Redis	Caching, session store	Redis 7
Kafka	Event streaming between services	Apache Kafka

3.5 Chosen Architecture Pattern and Rationale

Microservices + Event-Driven + BPMN Orchestration chosen for:

- Independent scaling of claims, quotes, and underwriting services
- Loose coupling via Kafka events
- BPMN engine externalizes workflow logic for easy updates without redeploying services
- Monolith rejected due to deployment rigidity and poor fault isolation

3.6 Technology Stack & Data Stores

Layer	Technology
Frontend	React 18, TypeScript, Redux Toolkit
Backend	Java 17, Spring Boot 3, Node.js (notifications)
Workflow	Camunda BPMN 2.0
Database	PostgreSQL 15, Redis 7
Messaging	Apache Kafka
Storage	S3-compatible object store
Infra	Docker, Kubernetes (EKS), Terraform, GitHub Actions
Monitoring	Prometheus, Grafana, ELK Stack

3.7 Risks & Mitigations

Risk	Mitigation
Workflow engine failure halts claims	HA cluster + persistent job store
Third-party KYC downtime	Fallback manual KYC queue + circuit breaker
Data breach of PII	Encryption at rest/in transit + RBAC + audit logs
Payment gateway failure	Retry queue + reconciliation job

3.8 Traceability to Requirements

R1 (Quote Generation) → Quote Service
R2 (Claims Processing) → Claims Service + Workflow Engine
R3 (Policy Issuance) → Policy Service
R4 (KYC Verification) → Underwriting Service

3.9 Security Architecture

Threat Modeling (STRIDE):

Threat	Mitigation
Spoofing	MFA + JWT + mutual TLS
Tampering	Signed documents, append-only audit logs
Repudiation	Digital signatures + non-repudiation logs
Information Disclosure	AES-256 at rest, TLS 1.2+ in transit, PII masking
Denial of Service	Rate limiting, WAF, autoscaling
Elevation of Privilege	RBAC + least privilege + secrets management (Vault)

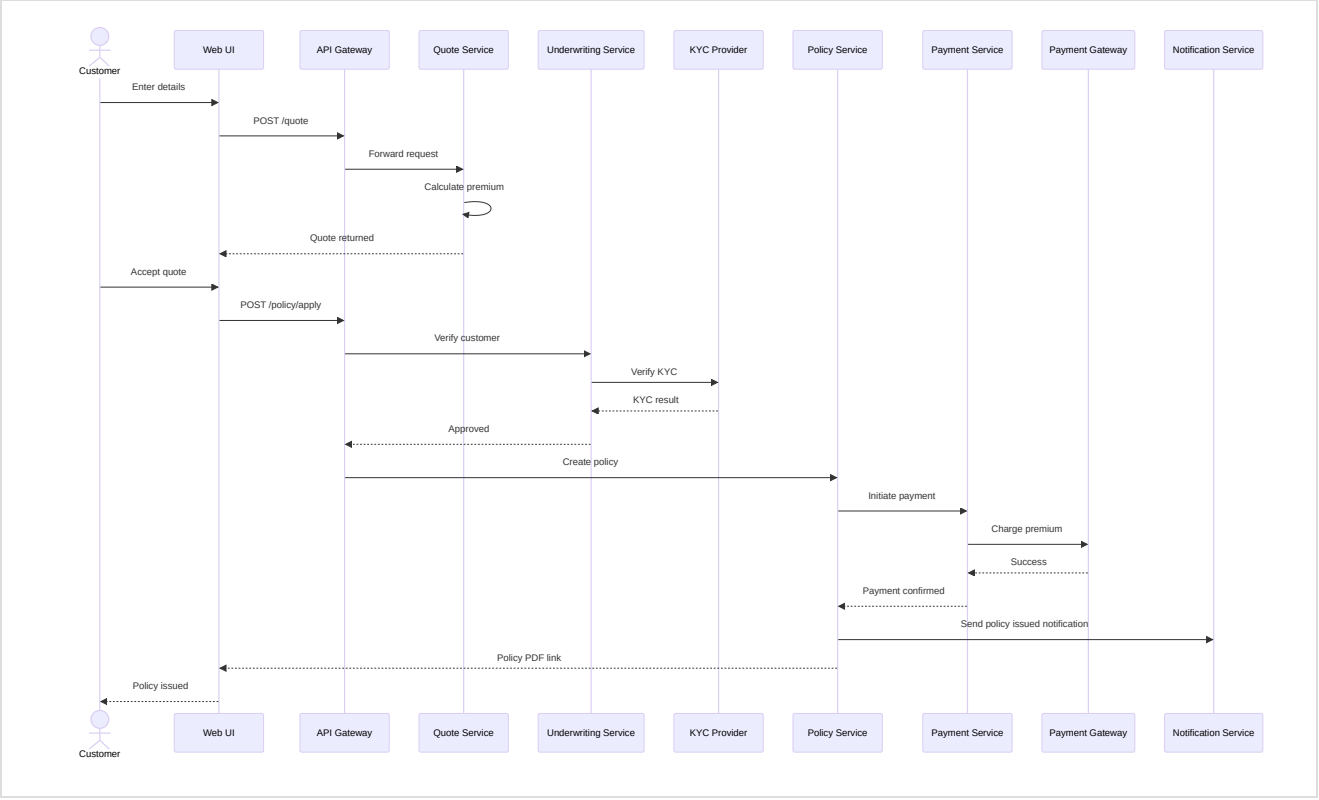
4. Design

4.1 Design Overview

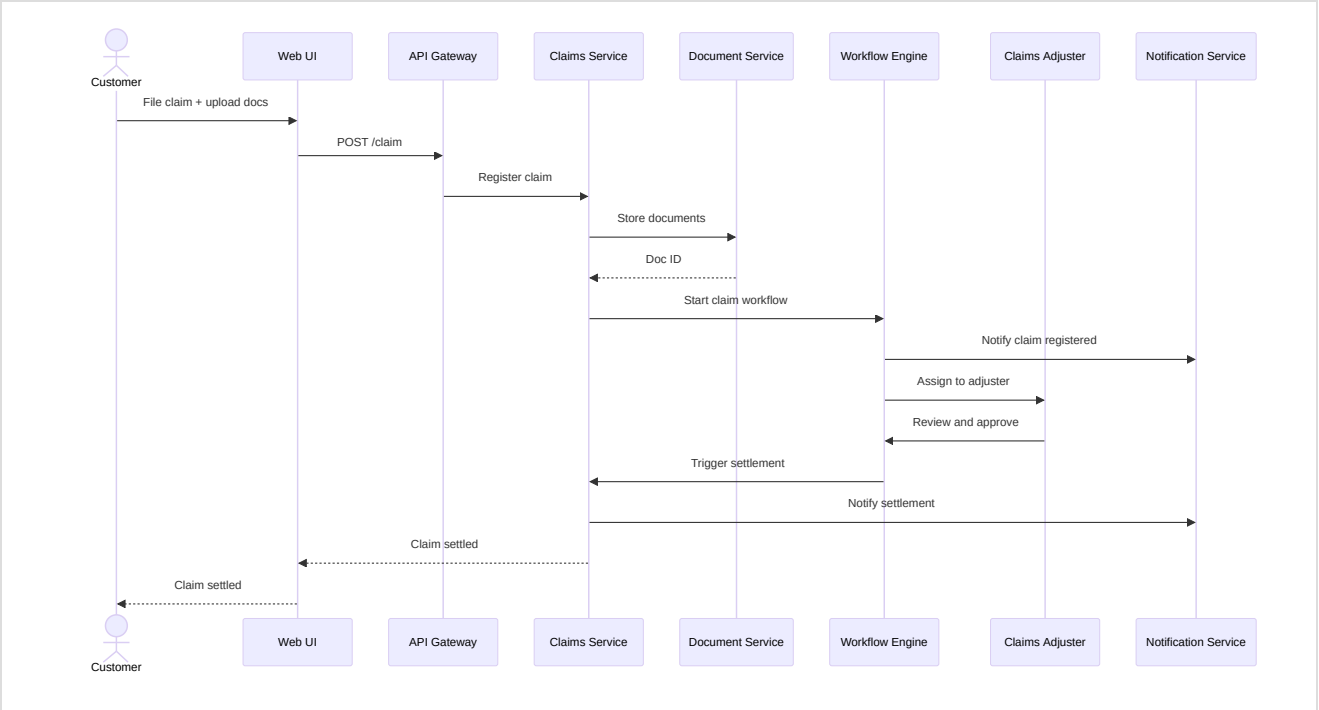
Modular microservices designed with clear bounded contexts. Each service owns its database schema. Inter-service communication via Kafka events; synchronous calls via REST with circuit breakers.

4.2 UML Sequence Diagrams

4.2.1 Sequence Diagram — Policy Purchase Flow



4.2.2 Sequence Diagram — Claim Processing Flow



4.3 API Design

4.3.1 Quote API

```
POST /api/v1/quote
Request:  { "age": 30, "income": 800000, "coverageType": "health", "term": 1 }
Response: { "quoteId": "Q123", "premium": 12500, "validUntil": "2025-10-01" }
Errors:   400 Invalid input, 503 Actuarial service unavailable
```

4.3.2 Claim API

```
POST /api/v1/claim
Request:  { "policyId": "P456", "claimType": "accident", "amount": 50000, "docs": ["doc1.p
Response: { "claimId": "C789", "status": "REGISTERED", "eta": "5 days" }
Errors:   401 Unauthorized, 404 Policy not found, 422 Invalid claim
```

4.4 Error Handling, Logging & Monitoring

- Standardized error envelope: `{ code, message, traceId }`
- No PII in logs; structured JSON logging
- Centralized logging via ELK; metrics via Prometheus + Grafana
- Alerts on: claim backlog > threshold, payment failure rate > 1%, uptime < 99.9%

4.5 UX Design

Accessible UI with WCAG 2.1 AA compliance. Multi-language (English, Hindi, regional). Step-by-step guided claim filing. Progress tracker for claims. Voice assistance for

accessibility.

4.6 Open Issues & Next Steps

- AI-based fraud detection (ML model integration)
- Blockchain-based policy audit trail
- Mobile native app (React Native)

5. Appendices

5.1 Glossary

IWAS, BPMN, KYC, RBAC, ADR, PCI-DSS, TLS, SLA, CI/CD.

5.2 References

IEEE 42010, OWASP ASVS, NIST SP 800-160, IRDAI guidelines.

5.3 Tools

PlantUML, draw.io, Swagger/OpenAPI, Camunda Modeler, GitHub Actions.